

D. Sudhakar
192524063Compiler
Design.

Text.

1. Consider the following while - statement -
 while $A > B$ and $A \leq 2 * B$ -s do $A := A + B$;

i) construct the Parse tree for the above

~~Parse~~ Given while statement.

ii) write the intermediate code for while - statement -

2. write down the translation scheme to generate code for assignment statement. list the scheme for generating three address code for the assignment statement $x = (a + b) * (c + d)$.

3. Consider the following code segment written in a high - level programming language.

Back Patching.

Given:

```

if (a > b) {
    x = x + z;
}
else {
    x = x - z;
}
  
```

87 - 1

8

a, Intermediate code

first generate the conditions :

1: if $a > b$ goto -

2: goto -

then the then block:

3: $t_1 = y + z$

4: $x = t_1$

5: goto -

then the else block.

6: $t_2 = y - z$

7: $x = t_2$

b) Back patching :

now assign labels.

true condition

if

$a > b$

is true, execution must go to the then block

∴

1. if $a > b$ goto 2.

(5)

false condition

If the condition is false, execution must go to the else block.

∴ 2: goto L4

then block

L2:

3: $t_1 = y + z$

4: $x = t_1$

5: goto L5

then conditional jump at instruction 5 skips the else block.

else block

L4:

6: $t_2 = y - z$

7: $x = t_2$

final label

L5:

final Three Address code .

1: if $a > b$ goto L2

2: goto L4

L2:

3: $t_1 = y + z$

4: $x = t_1$

5: goto L5

L4:

6: $t_2 = y - z$

7: $x = t_2$

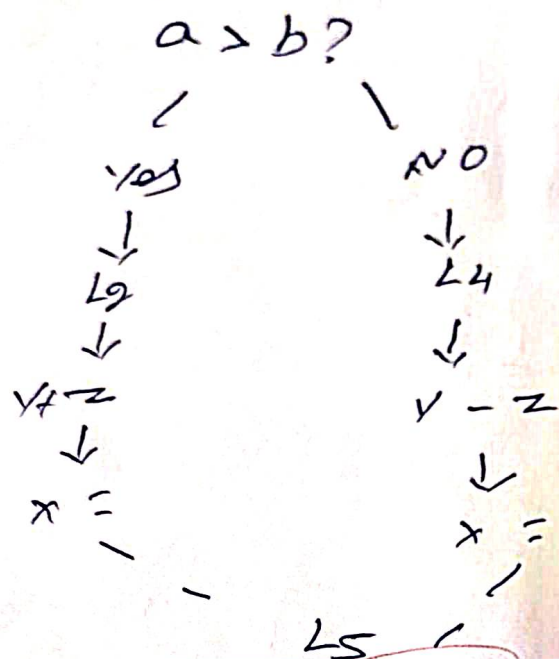
L5:

Back Patching Table:

Instruction	Initially	Backpatched to
1	if $a > b$ goto -	L2
2	goto -	L4
3	goto -	L5

Back Patching is used when the target address of a jump is not known at the time instruction is initially generated.

Control flow:

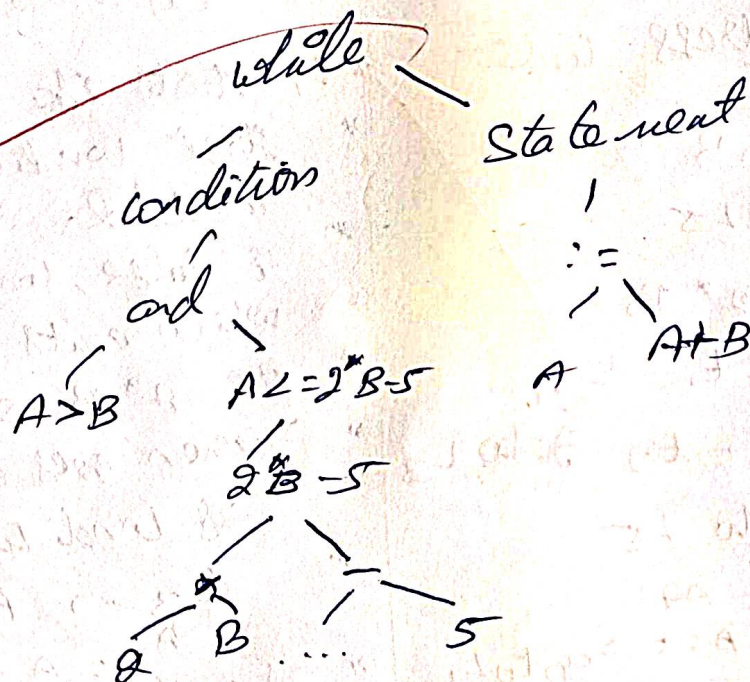


while statement:

while $A > B$ and $A \leq 2 * B - 5$ do $A := A + B$;

1) Parse tree.

the logical structure :



Detailed execution structure :

while

condition : $A \wedge D$

$$A > B$$

$$A \leq 2 * B - 5$$

$$2 * B$$

$$(2 * B) - 5$$

Body

$$A := A + B$$

Operator evaluation :

- * It has the higher precedence than - .
- * Therefore $2 * B - 5$ is evaluated as $(2 * B) - 5$.
- * The two relational expressions are the combined using AND .

Intermediate code :	Explanation
Three - address code :	
L1: if $A > B$ goto L2 goto L5	* L1 starts the while loop * first condition $A > B$ is checked. * if false, control goes to L5, which exits the loop.
L2: $t_1 = 2 * B$ $t_2 = t_1 - 5$	* The $2 * B - 5$ is calculated.
if $A \leq t_2$ goto L3 goto L5	* then goes to L3 * if condition $A \leq t_2$
L3: $t_3 = A + B$ $A = t_3$	* $(L3 \Rightarrow) A + B$ $t_3 = A$ $A := A + B$
L5: goto L1	* L5 not repeat the loop.

$$x = (a+b) * (c+d)$$

Translation scheme.

$$S \rightarrow id = E$$

for binary

$$E \rightarrow E_1 + E_2$$

$$E.place = new temp$$

$$emit (E.place = E_1.place + E_2.place)$$

$$E \rightarrow E_1 - E_2$$

$$E.place = new temp$$

$$emit (E.place = E_1.place - E_2.place)$$

$$E \rightarrow E_1 * E_2$$

$$E.place = new temp$$

$$emit (E.place = E_1.place * E_2.place)$$

$$E \rightarrow E_1 / E_2$$

$$E.place = new temp$$

$$emit (E.place = E_1.place / E_2.place)$$

$$S \rightarrow id = E$$

$$x = (a+b) * (c+d)$$

$$t_1 = (a+b)$$

$$t_2 = (c+d)$$

$$t_3 = t_1 * t_2$$

$$x = t_3$$

$$x = (a+b) * (c+d)$$

$$(a+b)$$

↓

$$t_1 = (a+b)$$

↓

$$t_2 = (c+d)$$

↓

$$t_3 = (c+d)$$

↓

$$t_3 = t_1 * t_2$$

↓

x

↓

$$t_3 = x$$